

鸿蒙工程接入

大约 18 分钟 约 5466 字

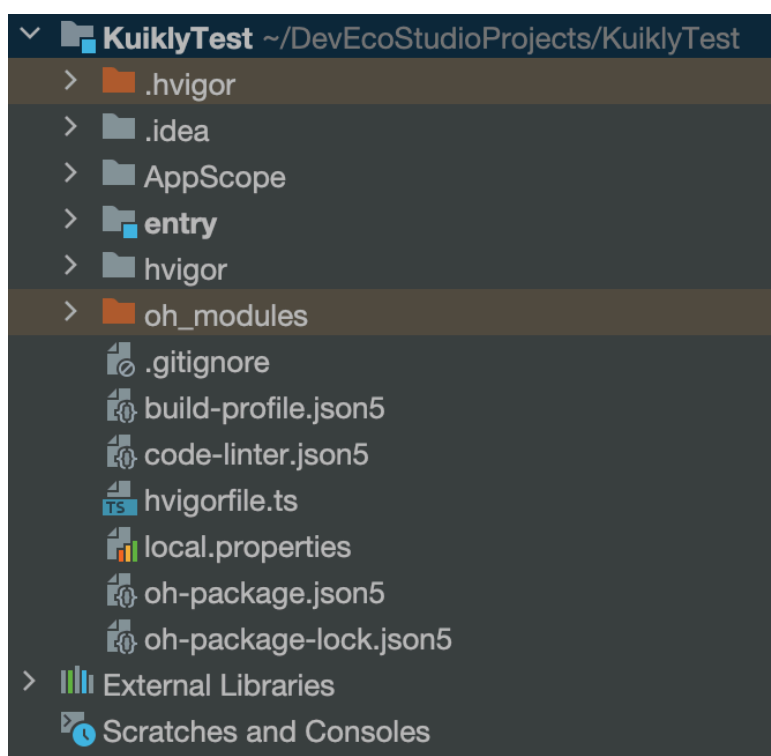
注意

1. 在此之前请确保已经完成**KMP侧 Kuikly**的接入，如还未完成，请移步[Kuikly KMP侧接入](#)
2. 鸿蒙模拟器不支持X86版的Mac，推荐使用Apple Silicon(Arm)版的Mac进行鸿蒙的开发

完成**Kuikly KMP**侧的配置后, 我们还需要将**Kuikly**渲染器和适配器接入到宿主平台中，此文档适用于您想把Kuikly渲染器接入到您现有的鸿蒙工程中。下面我们来看下，如何在现有鸿蒙工程中接入Kuikly渲染器。

我们用鸿蒙开发IDE DevEco Studio新建一个名为**KuiklyTest**的新工程并假设这个工程是你现有的鸿蒙工程：

反馈文档建议



添加Kuikly渲染器依赖

编辑entry模块的oh-package.json5，添加 Kuikly 相关 dependencies 依赖项：

```
// entry/oh-package.json5
{
  ...
  "dependencies": {
    ...
    "@kuikly-open/render": 'KUIKLY_RENDER_VERSION'
  }
}
```

json

提示

- KUIKLY_RENDER_VERSION 需要替换为实际的 kuikly 版本号，在这里[查看最新版本](#)
- 版本号需要和[KMP跨端工程](#)保持一致

点击右上角【Sync Now】（或者在entry目录下命令行执行ohpm install）。

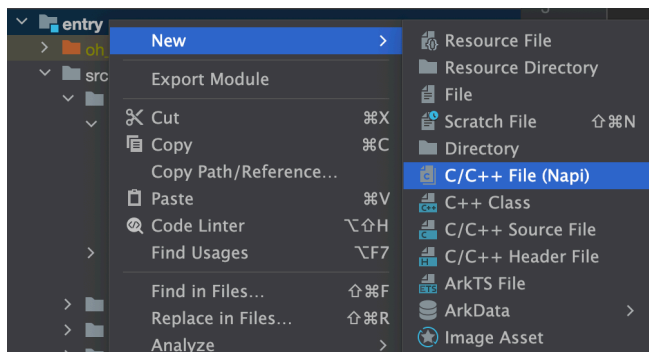
反馈
文档
建议

建鸿蒙运行时初始化接口

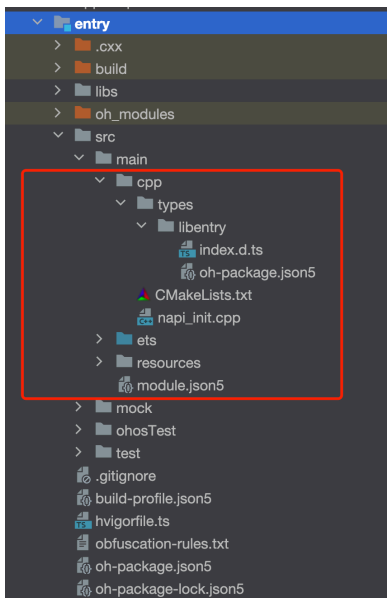
Kuikly 鸿蒙端渲染是基于ArkUI C-API 实现，在业务接入时，需要通过 NAPI ，将运行时初始化接口暴露到业务ArkTS层。

添加C++（NAPI）支持

在鸿蒙工程的 entry 入口模块添加C++（NAPI）支持（以前加过的跳过）。右键点击 entry 目录，在弹出的菜单中做如下选择：



确认创建后，会在 entry 目录新增 C++代码目录，如下图：



添加 NAPI 初始化入口函数

在上述C++目录下的**napi_init.cpp**文件，添加**InitKuikly**初始化入口，并暴露给ArkTS。具体实现代码，请参考源码工程 `core-render-ohos/entry` 模块的**napi_init.cpp**类。

反馈
文档
建议

C++

```
// entry/src/main/cpp/napi_init.cpp
#include "napi/native_api.h"

static napi_value InitKuikly(napi_env env, napi_callback_info info) {
    // 添加业务代码初始化逻辑。具体见后续步骤说明
    return nullptr;
}

EXTERN_C_START
static napi_value Init(napi_env env, napi_value exports)
{
    napi_property_descriptor desc[] = {
        // 导出 initKuikly，使其可以被ArkTS层访问和调用
        {"initKuikly", nullptr, InitKuikly, nullptr, nullptr, nullptr,
napi_default, nullptr},
    };
    napi_define_properties(env, exports, sizeof(desc) / sizeof(desc[0]), desc);
    return exports;
}
EXTERN_C_END

static napi_module demoModule = {
    .nm_version = 1,
```

```

        .nm_flags = 0,
        .nm_filename = nullptr,
        .nm_register_func = Init,
        .nm_modname = "entry",
        .nm_priv = ((void*)0),
        .reserved = { 0 },
    };

extern "C" __attribute__((constructor)) void RegisterEntryModule(void)
{
    napi_module_register(&demoModule);
}

```

在C++目录下的`index.d.ts`文件，对`initKuikly`进行接口声明

```

// entry/src/main/cpp/types/libentry/index.d.ts
export const initKuikly: () => number;

```

ts

反馈
文档
建议

关联NativeManager

在`entry/src/main/ets`下创建`kuikly`目录。

在`kuikly`目录下，创建`MyNativeManager.ets`类，实现`KuiklyNativeManager`类的`loadNative`接口，将Kuikly运行时初始化入口与框架`KuiklyNativeManager`关联。请参考源码工程 `core-render-ohos/entry` 模块的`MyNativeManager.ets`类。

```

// entry/src/main/ets/kuikly/MyNativeManager.ets
import { KuiklyNativeManager } from '@kuikly-open/render';
import Napi from 'libentry.so';

class MyNativeManager extends KuiklyNativeManager {
    protected loadNative(): number {
        // 调用Napi接口，初始化 Kuikly Native
        return Napi.initKuikly();
    }
}

// 导出一个全局的 KuiklyNativeManager 实例给 Kuikly 页面共用
const globalNativeManager = new MyNativeManager();

export default globalNativeManager;

```

ts

实现Kuikly承载容器

创建委托类

kuikly目录下，创建**IKuiklyViewDelegate**委托者实现类**KuiklyViewDelegate.ets**，用于向框架注册自定义View和Module、框架感知页面生命周期等。
请参考源码工程 core-render-ohos/entry 模块的**KuiklyViewDelegate.ets**类，注入自定义View 和 Module。

```
// entry/src/main/ets/kuikly/KuiklyViewDelegate.ets
import { IKuiklyViewDelegate, KRRenderModuleExportCreator,
KRRenderViewExportCreator } from '@kuikly-open/render';

export class KuiklyViewDelegate extends IKuiklyViewDelegate {
  getCustomRenderViewCreatorRegisterMap(): Map<string, KRRenderViewExportCreator> {
    const map: Map<string, KRRenderViewExportCreator> = new Map();
    return map;
  }

  getCustomRenderModuleCreatorRegisterMap(): Map<string,
KRRenderModuleExportCreator> {
    const map: Map<string, KRRenderModuleExportCreator> = new Map();
    return map;
  }
}
```

反
馈
文
档
建
议

ts

委托类说明

可以重写相关方法，实现自定义、扩展、配置 Kuikly 等功能。

```
export abstract class IKuiklyViewDelegate extends KRNativeRenderController {
  /**
   * 获取自定义扩展渲染视图创建注册Map
   */
  abstract getCustomRenderViewCreatorRegisterMap(): Map<string,
KRRenderViewExportCreator>;
```

ts

```

/**
 * 获取自定义扩展渲染视图创建注册Map。
 * 通过这种方式注册的creator，创建的自定义view将不会有影子节点处理基础事件，需要用户
在arkts侧响应所有属性的设置。
 * 当这个方式的好处是，由于不存在影子节点，其view曾经和DSL中定义的是保持一致的。
 * 建议仅在有强一致层级需求的时候才采用。
 */
getCustomRenderViewCreatorRegisterMapV2(): Map<string,
KRRenderViewExportCreator>{
    // by default
    return new Map();
}

/**
 * 获取自定义扩展module创建注册Map
 */
abstract getCustomRenderModuleCreatorRegisterMap(): Map<string,
KRRenderModuleExportCreator>;

/**
 * 获取自定义[KuiklyRenderView]生命周期回调
 */
getKuiklyRenderViewLifecycleCallback(): IKuiklyRenderViewLifecycleCallback |
null {
    return null
}

/**
 * Kuikly框架设置性能监控选项，默认只开启动监控
 * @return Array<KRMonitorType>: 需要设置的性能监控选项列表(目前仅支持启动监控)
 */
performanceMonitorTypes(): Array<KRMonitorType> {
    return [KRMonitorType.LAUNCH];
}

/**
 * 回调启动数据
 */
onGetLaunchData(data: Record<string, number>): void {
}

/**
 * 回调性能数据

```

```

    */
    onGetPerformanceData(data: Record<string, number>): void {
    }

    /**
     * 字体缩放是否跟随系统
     * @return boolean true:跟随系统(默认) false:不跟随系统(缩放比例为1)
     */
    fontSizeScaleFollowSystem(): boolean {
        return true
    }
}

```

实现Kuikly承载容器

在page页面容器中加入Kuikly组件（以 pages/Index 为例，也可以是新建的page），触发Kuikly页面加载。

反馈文档建议
Kuikly组件参数说明

- `pageName` : 页面名称，对应@Page注解中定义的名称
- `pageData` : 页面数据，传递给Kuikly页面的参数
- `delegate` : 委托者实现，用于注册自定义View和Module
- `initialSize` : 初始尺寸设置，用于指定Kuikly容器的初始宽高（可选参数）
 - 格式： { width: number, height: number }
 - 用途：初始化时传入正确的容器尺寸，可以提前跨端页面的创建（传入错误值会导致重复排版和布局跳变）
- `onControllerReadyCallback` : 控制器就绪回调
- `nativeManager` : 原生管理器实例

请参考源码工程 core-render-ohos/entry 模块的**Index.ets**类。

```

// entry/src/main/ets/kuikly/pages/Index.ets
import { KRRecord, KRNativeRenderController, Kuikly } from '@kuikly-open/render';
import globalNativeManager from '../kuikly/MyNativeManager';
import { KuiklyViewDelegate } from '../kuikly/KuiklyViewDelegate';
import router from '@ohos.router';
import { hilog } from '@kit.PerformanceAnalysisKit';

```

ts

```
import { ContextCodeHandler } from '../kuikly/ContextCodeHandler';

@Entry
@Component
struct Index {
  private kuiklyViewDelegate = new KuiklyViewDelegate();
  private kuiklyController : KRNativeRenderController | null = null

  private pageName: string | null = null;
  private pageData?: KRRRecord;
  private contextCode: string = '';
  private contextCodeHandler: ContextCodeHandler = new ContextCodeHandler();
  private useDefaultBackPress = true
  @State showKuikly: boolean = false;

  onBackPress(): boolean | void {
    if(this.useDefaultBackPress){
      return
    }

    if(this.kuiklyViewDelegate){
      this.kuiklyController?.sendBackPressed()
      return true
    }
  }

  aboutToAppear(): void {
    const params = router.getParams() as Record<string, Object>;
    this.pageName = params?.pageName as string;
    this.pageData = (params?.pageData as KRRRecord | null) ?? {}
    if (this.contextCodeHandler.isNeedGetContextCode(params)) {
      this.contextCodeHandler.handleGetContextCode(getContext(), params,
(contextCode) => {
        this.contextCode = contextCode;
        this.showKuikly = true;
      }, (stack) => {
        this.showExceptionDialog(stack);
      })
    } else {
      this.showKuikly = true;
    }
  }

  build() {
```



```
Stack() {
  if (this.showKuikly) {
    Kuikly({
      pageName: this.pageName ?? 'router',
      pageData: this.pageData ?? {},
      delegate: this.kuiklyViewDelegate,
      contextCode: this.contextCode,
      executeMode: this.contextCodeHandler.getExecuteMode(this.contextCode),
      // 可选: 设置Kuikly容器的初始尺寸
      // initialSize: { width: this.calculateWidth(), height:
this.calculateHeight() },
      onControllerReadyCallback: (controller) => {
        this.kuiklyController = controller
        controller.registerExceptionCallback((executeMode, stack) => {
          this.showExceptionDialog(stack);
          const stackInfo: KRRRecord = JSON.parse(stack)
          stackInfo['stack'].toString().split('\n').forEach((it) => {
            hilog.error(0x0000, 'demo', '%{public}s', it);
          })
        });
      },
      nativeManager: globalNativeManager,
    })
  }
}.expandSafeArea([SafeAreaType.KEYBOARD])
// .backgroundColor(Color.Green)
}

private showExceptionDialog(stack: string) {
  // 对话框显示异常堆栈
}

onPageShow(): void {
  const res = getContext(this).resourceDir
  this.kuiklyViewDelegate.pageDidAppear()
}

onPageHide(): void {
  this.kuiklyViewDelegate.pageDidDisappear()
}
}
```

```
// entry/src/main/ets/kuikly/pages/ContextCodeHandler.ets
import { KRRenderExecuteModeBase, KRRenderNativeMode } from '@kuikly-open/render';

export class ContextCodeHandler {
  isNeedGetContextCode(params: Record<string, Object>) {
    return false
  }

  handleGetContextCode(context: Context, params: Record<string, Object>,
callback: (contextCodeParam: string) => void,
  exceptionCallback: (stack: string) => void) {

  }

  getExecuteMode(contextCode: string): KRRenderExecuteModeBase {
    return KRRenderNativeMode.Native
  }
}
```

ts

反馈文档建议

承载容器作为组件批量嵌入页面

在鸿蒙侧， `Kuikly()` 组件本身就是一个 ArkUI 组件，天然支持嵌入到任意 Native 布局容器中。与页面级使用的区别在于：页面级直接在 `@Entry` 页面的 `build()` 中放置一个全屏的 `Kuikly()` 组件，而 View 粒度则是在一个 Native 布局容器（如 `WaterFlow`、`List`、`Grid` 等）中嵌入多个 `Kuikly()` 组件。详细的操作步骤可参考以下代码：

代码示例

```
import globalNativeManager from '../kuikly/MyNativeManager';
import { KuiklyViewDelegate } from '../kuikly/KuiklyViewDelegate';
import { AppKRRenderManager } from '../kuikly/adapters/AppKRRenderManager';
import { KRNativeRenderController, KRRenderNativeMode, KRRecord, Kuikly } from '@kuikly-open/render';

@Entry
@Component
struct NativeContainerPage {
  aboutToAppear(): void {
    AppKRRenderManager.getInstance().initIfNeed();
  }
}
```

ts

```

}

build() {
  Column() {
    // 在原生布局中嵌入一个 Kuikly 子视图
    Kuikly({
      pageName: 'YourPageName',
      pageData: { 'key': 'value' } as KRRRecord,
      delegate: new KuiklyViewDelegate(),
      contextCode: '',
      executeMode: KRRRenderNativeMode.Native,
      nativeManager: globalNativeManager,
      initialSize: { width: 200, height: 300 }
    })
    .width(200)
    .height(300)
  }
  .width('100%')
  .height('100%')
}

```

反馈文档建议

列表场景中使用时，推荐配合 @Reusable 组件：

```

@Reusable
@Component
struct KuiklyListItem {
  @State itemData: Record<string, Object> = {};
  private delegate = new KuiklyViewDelegate();

  build() {
    Column() {
      Kuikly({
        pageName: 'YourCardPage',
        pageData: this.itemData as KRRRecord,
        delegate: this.delegate,
        contextCode: '',
        executeMode: KRRRenderNativeMode.Native,
        nativeManager: globalNativeManager,
        initialSize: { width: 180, height: 200 }
      })
    }
  }
}

```

ts

```

        .width('100%')
        .height(200)
    }
}
}

```

完整的 View 粒度接入实践示例（卡片式风格瀑布流），请参考源码工程 ohosApp 模块的 NativeAppWaterfall.ets 页面，该 Demo 使用 WaterFlow + LazyForEach 实现两列瀑布流，每个 FlowItem 中各嵌入了一个 Kuikly() 组件。另外也可参考 KuiklyInList.ets 了解在 List 容器中的嵌入方式。

与页面级方式的主要不同点

	页面级 (@Entry Page)	View 粒度 (嵌入容器)
容器	Kuikly() 作为页面唯一根组件	多个 Kuikly() 组件嵌入到 WaterFlow 、 List 等原生容器中
生命周期	由 @Entry 页面的 onPageShow / onPageHide 自动触发 delegate 的 pageDidAppear / pageDidDisappear	由宿主 Native 页面手动管理，或借助 @Reusable 组件的 aboutToAppear / aboutToDisappear
initialSize	可选（组件自动获取页面尺寸）	建议指定 ，传入正确的初始宽高可以提前跨端页面的创建，避免重复排版
delegate	每个页面一个 delegate 实例	每个嵌入的 Kuikly() 组件各自拥有一个 delegate 实例
组件复用	无需关注	使用 @Reusable + LazyForEach 优化列表性能

注意事项

- initialSize 建议指定：**View 粒度嵌入时， Kuikly() 组件的初始尺寸不像全屏页面那样确定，建议通过 initialSize: { width: xxx, height: xxx } 传入正确的初始尺寸
- pageData 扁平传递：**pageData 传入扁平的 KRRRecord 对象即可（如 { 'key': 'value' } ），框架内部会自动将其包裹在 param key 下

3. **delegate 独立实例**: 每个 `Kuikly()` 组件应各自持有独立的 `KuiklyViewDelegate` 实例
4. **列表场景推荐 @Reusable**: 在 `List` / `WaterFlow` 等列表容器中使用 `@Reusable` 组件和 `LazyForEach` 实现组件复用, 提升性能

注意

view 粒度接入进的原生页面, 还要在 `AppKRRouterAdapter.ets` 中添加 `pageName` 路由分支, 并在 `main_pages.json` 中注册新建的 ArkTS Page 路径。

实现适配器（必须实现部分）

Kuikly 框架为了灵活和可拓展性, 不会内置实现异常处理, 日志实现等功能, 而是通过适配器的设计模式, 将具体实现委托给宿主App实现。

Kuikly 为鸿蒙端宿主工程提供了以下适配器, 需宿主平台按需实现

1. 日志适配器: 用于给Kuikly框架和Kuikly业务实现日志打印。推荐宿主侧实现
2. 页面路由适配器: 用于实现跳转到Kuikly容器。宿主侧必须实现
3. PAG加载适配器: 用于给Kuikly提供PAG加载的能力。**宿主按需实现**（使用PAG组件时必须实现, 可参考[AppKRPAGAdapter.ets \(https://github.com/Tencent-TDS/KuiklyUI/blob/main/ohosApp/entry/src/main/ets/kuikly/adapters/AppKRPAGAdapter.ets\)](https://github.com/Tencent-TDS/KuiklyUI/blob/main/ohosApp/entry/src/main/ets/kuikly/adapters/AppKRPAGAdapter.ets)）

日志适配器示例

请参考源码工程 `core-render-ohos/entry` 模块的 `AppKRLogAdapter.ets` 类。

```
// entry/src/main/ets/kuikly/adapters/AppKRLogAdapter.ets
import { IKRLogAdapter } from '@kuikly-open/render';
import { hilog } from '@kit.PerformanceAnalysisKit';

export class AppKRLogAdapter implements IKRLogAdapter {

  i(tag: string, msg: string): void {
    hilog.info(0x30, tag, '%{public}s', msg)
  }

  d(tag: string, msg: string): void {
    hilog.debug(0x30, tag, '%{public}s', msg)
  }
}
```

ts

```

    }

    e(tag: string, msg: string): void {
        hilog.error(0x30, tag, '%{public}s', msg)
    }

}

```

路由适配器示例

请参考源码工程 core-render-ohos/entry 模块的**AppKRRouterAdapter.ets**类。

```

// entry/src/main/ets/kuikly/adapters/AppKRRouterAdapter.ets
import { KRRRecord } from '@kuikly-open/render';
import { IKRRouterAdapter } from '@kuikly-open/render';
import router from '@ohos.router';
import { common } from '@kit.AbilityKit';

export class AppKRRouterAdapter implements IKRRouterAdapter {
    openPage(context: common.UIAbilityContext, pageName: string, pageData:
KRRRecord): void {
        router.pushUrl({
            url: 'pages/Index',
            params: {
                pageName,
                pageData
            }
        })
    }

    closePage(context: common.UIAbilityContext): void {
        router.back()
    }
}

```

ts

反馈
文档
建议

初始化适配器

在 UIAbility 的 onWindowStageCreate 时机初始化 Kuikly（多ability场景可以把初始化时机提前到AbilityStage，避免相互覆盖）：

请参考源码工程 core-render-ohos/entry 模块的**EntryAbility.ets**类。

```
// entry/src/main/ets/entryability/EntryAbility.ets
import { KuiklyRenderAdapterManager } from '@kuikly-open/render';

export default class EntryAbility extends UIAbility {
    ...
    onWindowStageCreate(windowStage: window.WindowStage): void {
        // Main window is created, set main page for this ability
        // 日志适配器
        KuiklyRenderAdapterManager.krLogAdapter = new AppKRLogAdapter();
        // 路由适配器
        KuiklyRenderAdapterManager.krRouterAdapter = new AppKRRouterAdapter();
        ...
    }
}
```

ts

反馈
文档
建议

链接Kuikly业务代码

Kuikly业务代码，在鸿蒙平台上会被编译成 so 产物，下面以本地so文件方式为例介绍链接 Kuikly 业务代码的流程。我们先前在 Kuikly [KMP跨端工程接入](#) 中已经新建了 Kuikly 业务工程，然后将这个业务工程的业务代码编译成的 .so 链接到我们的现有鸿蒙工程。

生成 so 产物和头文件

鸿蒙Kuikly业务代码编译生成 so 产物，详细步骤参考[鸿蒙平台开发方式](#)。

[KMP侧接入](#)工程中，编译跨端工程的 shared 模块，命令行执行 `./gradlew -c settings.ohos.gradle.kts :shared:linkOmosArm64` 编译鸿蒙so产物。

拷贝Kuikly业务代码产物

将业务代码生成的动态链接库文件libshared.so和头文件libshared_api.h拷贝到C++模块中：



反馈
文档
建议

修改CMakeList

修改C++目录下 CMakeLists.txt，导入业务产物和Kuikly SDK的动态链接库：

```
set(NATIVERENDER_ROOT_PATH ${CMAKE_CURRENT_SOURCE_DIR})

# Kuikly SDK
add_library(kuikly_render ALIAS render::kuikly)
# 业务产物
add_library(kuikly_shared SHARED IMPORTED)
set_target_properties(kuikly_shared
    PROPERTIES
```

cmake

IMPORTED_LOCATION

```
${NATIVERENDER_ROOT_PATH}/../../../../libs/${OHOS_ARCH}/libshared.so)
# 追加「kuikly_shared」和「kuikly_render」到入口模块target_link_libraries
target_link_libraries(entry PUBLIC libace_napi.z.so kuikly_shared kuikly_render)
```

实现 NAPI 初始化入口函数InitKuikly

在前述章节[添加 NAPI 初始化入口函数](#)步骤，我们创建了**InitKuikly**初始化入口函数，我们在这个步骤实现这个函数即可。

反馈文档建议

```
// entry/src/main/cpp/napi_init.cpp
#include "libshared_api.h"
#include "napi/native_api.h"

static napi_value InitKuikly(napi_env env, napi_callback_info info) {
    // symbols入口名和kuikly工程的配置有关，具体查看产物的头文件
    auto api = libshared_symbols();
    int handler = api->kotlin.root.initKuikly();
    napi_value result;
    napi_create_int32(env, handler, &result);
    return result;
}

EXTERN_C_START
static napi_value Init(napi_env env, napi_value exports) {
    napi_property_descriptor desc[] = {
        {"initKuikly", nullptr, InitKuikly, nullptr, nullptr, nullptr, napi_default,
        nullptr},
    };
    napi_define_properties(env, exports, sizeof(desc) / sizeof(desc[0]), desc);
    return exports;
}

EXTERN_C_END
```

C++

拷贝资源文件

如果有资源文件，需要把assets目录下的资源文件拷贝到entry/src/main/resources/resfile，例如：

```
shared/src/commonMain/assets/common/* ->
entry/src/main/resources/resfile/common/*
```

text

编写TestPage验证

完成上述步骤后, 我们便完成了Kuikly的接入。下面我们在[KMP侧接入](#)工程中的 shared 模块下新建页面名为test的 TestPage 进行测试。

```
@Page("test")
class TestPage : Pager(){
    override fun body(): ViewBuilder {
        return {
            attr {
                allCenter()
            }

            Text {
                attr {
                    fontSize(18f)
                    text("Hello Kuikly")
                    color(Color.GREEN)
                }
            }
        }
    }
}
```

kt

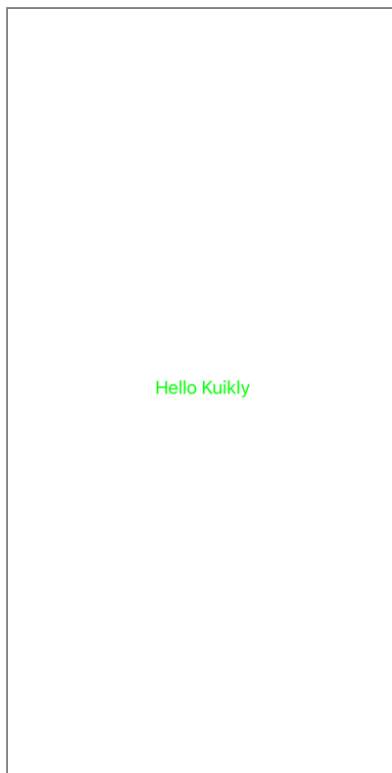
反馈
文档
建议

将 test 替换 router 作为 pageName 传入page/Index中, 指定跳转到我们刚新建的 TestPage 页面

```
// entry/src/main/ets/kuikly/pages/Index.ets
...
Kuikly({
    pageNumber: this.pageName ?? 'test',
    ...
})
...
```

ts

参考[生成 so 产物和头文件](#)、[拷贝Kuikly业务代码产物](#)，在KMP工程重新生成 so 产物和头文件，更新到鸿蒙工程，编译鸿蒙应用。当手机出现以下界面时,说明已经成功接入Kuikly



反馈
文档
建议

现适配器（按需实现部分）

图片加载适配器示例

该适配器用于给Kuikly的Image组件实现自定义图片加载能力，非必须实现, 业务可根据实际使用需求来决定是否实现。

接口定义于 Kuikly.h:

```
/**
 * @brief 业务图片加载完成后，用于回调给kuikly的函数指针
 * @param context 上下文
 * @param src image组件设置的src属性
 * @param image_descriptor 解码好的图片
 * @param new_src 新的src地址，比如从原src映射到一个新的src路径
 * @discuss 当image_descriptor非空时，kuikly优先用image_descriptor，其次再使用
new_src
 */
typedef void (*KRSetImageCallback)(const void* context,
                                   const char *src,
```

C

```

ArkUI_DrawableDescriptor *image_descriptor,
const char *new_src);

/**
 * @brief 自定义image adapter V2
 * @param context 上下文
 * @param src image组件设置的src属性
 * @param callback 自定义加载图片完成后可通过callback指针回调给kuikly，并把context以及src参数回填
 * @return 已处理则返回1，否则返回0
 */
typedef int32_t (*KRIImageAdapterV2)(const void *context,
const char *src,
KRSetImageCallback callback);

/**
 * @brief 自定义image adapter V3，新增 imageParams 参数
 * @param context 上下文
 * @param src image组件设置的src属性
 * @param imageParams 图片加载参数，JSON格式字符串，可为nullptr
 * @param callback 自定义加载图片完成后可通过callback指针回调给kuikly，并把context以及src参数回填
 * @return 已处理则返回1，否则返回0
 */
typedef int32_t (*KRIImageAdapterV3)(const void *context,
const char *src,
KRAnyData *imageParams,
KRSetImageCallback callback);

void KRRegisterImageAdapterV2(KRIImageAdapterV2 adapter);
void KRRegisterImageAdapterV3(KRIImageAdapterV3 adapter);

```

3 新增能力

KRIImageAdapterV3 相比 V2 新增了 imageParams 参数，用于接收 Kotlin 侧通过在src中所增加的自定义参数（如鉴权信息、加载策略等）。参数以 JSONObject 格式传入，在 KRIImageAdapterV3 传入为Map类型

使用方法

1. 确认CMakeList已链接kuikly_render

如已配置可跳过，链接方法参考上文[链接Kuikly业务代码](#)

```
target_link_libraries(
    .....
    kuikly_render
)
```

cmake

2. 头文件引入

在调用 KRRegisterImageAdapterV2 或 KRRegisterImageAdapterV3 的源文件中增加 include。如在 C++ 目录下的 **napi_init.cpp** 文件中 include 如下头文件：

```
#include <Kuikly/Kuikly.h>
```

3. Adapter实现

```
// entry/src/main/cpp/napi_init.cpp
#include <Kuikly/Kuikly.h>

// V2 实现（不需要 imageParams）
static int32_t MyImageAdapterV2(const void *context, const char *src,
    RSetImageCallback callback) {
    // 自定义图片加载逻辑
    // ...
    return 0;
}

// V3 实现（需要 imageParams）
static int32_t MyImageAdapterV3(const void *context, const char *src, KRAnyData
    *imageParams, RSetImageCallback callback) {

    // 获取imageParams,跨端侧传入的是: {"test":"abc"}
    std::map<std::string, std::string> paramsMap;
    // 方式1: 使用 KRAnyDataVisitMap 遍历所有参数（推荐）
    if (imageParams != nullptr && KRAnyDataIsMap(imageParams)) {
        // 定义 Lambda 作为访问器
        auto visitor = [](const char* key, KRAnyData value, void* userData) {
            auto* map = static_cast<std::map<std::string, std::string*>
            (userData);
            // 根据类型转换成字符串存储
            if (KRAnyDataIsString(value)) {
                const char* str = KRAnyDataGetString(value);
                if (str) {
                    (*map)[key] = str;
                }
            }
        };
        KRAnyDataVisitMap(imageParams, visitor, paramsMap);
    }
}
```

C

反馈
文档
建议

```

    }
    } else if (KRAnyDataIsInt(value)) {
        int32_t intVal;
        KRAnyDataGetInt(value, &intVal);
        (*map)[key] = std::to_string(intVal);
    } else if (KRAnyDataIsLong(value)) {
        int64_t longVal;
        KRAnyDataGetLong(value, &longVal);
        (*map)[key] = std::to_string(longVal);
    } else if (KRAnyDataIsFloat(value)) {
        float floatVal;
        KRAnyDataGetFloat(value, &floatVal);
        (*map)[key] = std::to_string(floatVal);
    } else if (KRAnyDataIsBool(value)) {
        bool boolVal;
        KRAnyDataGetBool(value, &boolVal);
        (*map)[key] = boolVal ? "true" : "false";
    }
};

// 遍历所有键值对
KRAnyDataVisitMap(imageParams, visitor, &paramsMap);
}

// 业务逻辑...
if (paramsMap.count("test") > 0) {
    auto value = paramsMap["test"];
    KR_LOG_INFO << "imageParams testxxx value: " << value;
}

// 方式2: 获取特定的参数值 (如果只需要某个字段)
if (imageParams != nullptr && KRAnyDataIsMap(imageParams)) {
    KRAnyData testValue = nullptr;
    if (KRAnyDataGetMapValue(imageParams, "test", &testValue) ==
KRANYDATA_SUCCESS && testValue != nullptr) {
        if (KRAnyDataIsString(testValue)) {
            const char *str = KRAnyDataGetString(testValue);
            KR_LOG_INFO << "imageParams test value: " << str;
        }
    }
}
}

```

```
// 自定义图片加载逻辑
// ...
return 0;
}
```

4. Adapter注册

可在使用 Kuikly 前进行 adapter 注册，作为示例，简单起见这里在 InitKuikly 中进行了注册，实际使用时可以在其他更早时机，也应该注意不要多次注册。

```
// entry/src/main/cpp/napi_init.cpp
static napi_value InitKuikly(napi_env env, napi_callback_info info) {
    // 二选一注册，V3 优先级高于 V2
    KRRegisterImageAdapterV2(MyImageAdapterV2);
    // 或
    KRRegisterImageAdapterV3(MyImageAdapterV3);

    // ...
}
```

反馈文档建议

完成后，可通过**模版工程**中的 **ImageAdapter基准测试** 页面来验证功能正常。

提示

鸿蒙端暂不支持capInset能力，请忽略 **ImageAdapter基准测试** 中的capInset测试项。

自定义字体适配器示例

该适配器非必须实现, 业务可根据实际使用需求来决定是否实现。

接口是KRRegisterFontAdapter，定义于Kuikly.h

```

/**
 * @brief 一个用于析构KRFonAdapterFontBufferForFamily返回的fontBuffer的回调函数
 * @param fontBuffer
 * @param len
 */
typedef void (*KRFonDataDeallocator)(char* data);

/**
 * @brief 根据fontFamily返回font family src或 font buffer.
 * 模式1: 返回font src, 这种情况输入是fontFamily, deallocator, 输出是函数返回值
 * 模式2: 通过参数指针返回font buffer, 这种情况输入是fontFamily, 输出是fontBuffer, len, deallocator
 * @param fontFamily
 * @param fontBuffer 用于返回font buffer指针
 * @param len 返回的font buffer长度
 * @param deallocator 用于释放font buffer指针或src, 如果是非空则kuikly使用完font buffer或src后会调用deallocator
 * @return font family
 */
typedef char* (*KRFonAdapter)(const char* fontFamily, char** fontBuffer, size_t* len, KRFonDataDeallocator* deallocator);

/**
 * @brief 注册font adapter
 * @param adapter adapter函数指针
 * @param fontFamily adapter对应的font family
 */
void KRRegisterFontAdapter(KRFonAdapter adapter, const char* fontFamily);

```

使用方法

1. 确认CMakeList已链接kuikly_render

如已配置可跳过，链接方法参考上文[链接Kuikly业务代码](#)

反馈文档建议

```

target_link_libraries(
    .....
    kuikly_render
)

```

cmake

2. 头文件引入

在调用KRRegisterFontAdapter的源文件中增加include。如在上述C++目录下的napi_init.cpp文件 include 如下头文件。

```
#include <Kuikly/Kuikly.h>
```

3. Adapter实现

具体实现代码，请参考源码工程 core-render-ohos/entry 模块的napi_init.cpp类。

```

// entry/src/main/cpp/napi_init.cpp
...
#include <Kuikly/Kuikly.h>
...
static char *MyFontAdapter(const char *fontFamily, char **fontBuffer, size_t
*len, KRFonDataDeallocator *deallocator) {
    if (isEqual(fontFamily, "Satisfy-Regular")) {

```

C


```

        return "rawfile:Satisfy-Regular.ttf";
    }
    return (char *)customFontPath.c_str();
}
...

```

4. Adapter注册 可在使用Kuikly前进行adapter注册，作为示例，简单起见这里在InitKuikly 中进行了注册，实际使用的时候可以在其他更早实际，也应该注意不要多次注册。

```

// entry/src/main/cpp/napi_init.cpp
...
static napi_value InitKuikly(napi_env env, napi_callback_info info) {
    KRRegisterFontAdapter(MyFontAdapter, "Satisfy-Regular");

    // ...
}
...

```

反馈
文档
建议

如何获得字体路径

一般通过网络等途径下载字体，这种情况下可以通过adapter返回路径即可。不过有的业务会将字体文件放到rawfile中，但目前还有没有稳定获取rawfile路径的方法，可参考demo片段把字体拷贝到临时文件目录中：

```

// copy font data to tmp folder
const content = getContext().resourceManager.getRawFileContentSync("Satisfy-Regular.ttf")
const destPath = `${getContext().tempDir}/Satisfy-Regular.ttf`;

fs.open(destPath, fs.OpenMode.CREATE | fs.OpenMode.READ_WRITE, (err:
BusinessError, data) => {
    if (err) {
        console.error("copy file failed with error message: " + err.message + ",
error code: " + err.code);
    } else {
        fs.write(data.fd, content.buffer, {offset: 0, length:
content.length}).then((result)=>{

```

```

        console.info(`copy file succeed:${result}`);
        Napi.setFontPath(destPath)
    })
}
})

```

并通过一个setFontPath接口设置给c++侧，让adapter返回：

此处NAPI调用设置可以参考[NAPI初始化逻辑](#)

反馈文档建议

```

static std::string customFontPath;

static napi_value SetFontPath(napi_env env, napi_callback_info info) {
    if (customFontPath.size() > 0) {
        return nullptr;
    }

    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);

    size_t length = 0;
    napi_status status;
    status = napi_get_value_string_utf8(env, args[0], nullptr, 0, &length);
    std::string buffer(length, 0);
    status = napi_get_value_string_utf8(env, args[0], reinterpret_cast<char*>
(buffer.data()), length + 1, &length);
    customFontPath = buffer;

    return nullptr;
}

static char *MyFontAdapter(const char *fontFamily, char **fontBuffer, size_t
*len, KRFontDataDeallocator *deallocator) {
    if (isEqual(fontFamily, "Satisfy-Regular")) {
        return "rawfile:Satisfy-Regular.ttf";
    }
    return (char *)customFontPath.c_str();
}

```

C

颜色值转换适配器

该适配器非必须实现, 业务可根据实际使用需求来决定是否实现。

接口是KRRegisterColorAdapter，定义于Kuikly.h

C++

```
/**
 * Color Adapter回调
 */
typedef int64_t (*KRCOLORAdapterParseColor)(const char* str);

/**
 * 注册c实现的颜色解析adapter，进程声明周期中，只应调用一次，建议在初始化阶段（如调用
 initKuikly前）进行调用。
 * example:
 * 1. Implement the adapter
 * static uint32_t MyColorParser(const char* str){
 *     uint32_t val = 0;
 *     ... parse from str ...
 *     return val;
 * }
 *
 * 2. Register before calling initKuikly
 * if(!registerd){// e.g. register could a static variable
 *     KRRegisterColorAdapter(&MyColorParser);
 * }
 */
void KRRegisterColorAdapter(KRCOLORAdapterParseColor adapter);
```

反馈
文档
建议

使用方法 使用方法和自定义字体适配器类似

```
target_link_libraries(
    .....
    kuikly_render
)
```

cmake

2. 头文件引入

在调用KRRegisterFontAdapter的源文件中增加include。如在上述C++目录下的 **napi_init.cpp** 文件 include 如下头文件。

```
#include <Kuikly/Kuikly.h>
```

3. Adapter实现

具体实现代码，请参考源码工程 core-render-ohos/entry 模块的 **napi_init.cpp** 类。

```
* 1. Implement the adapter
static int64_t MyColorAdapter(const char* str){
    // Add custom parsing and return actual color value.
    // Demo only returns -1 to allow kuikly automatically convert the color
    string
    return -1;
}
```

4. Adapter注册 可在使用Kuikly前进行adapter注册，作为示例，简单起见这里在 InitKuikly 中进行了注册，实际使用的时候可以在其他更早实际，也应该注意不要多次注册。

```
反馈  
文档  
建议  
// entry/src/main/cpp/napi_init.cpp
...
static napi_value InitKuikly(napi_env env, napi_callback_info info) {
    KRRegisterColorAdapter(MyColorAdapter);
    // ...
}
...
```

实现 PAG 适配器

与字体、图片适配器的定位不同，PAG 适配器是以 工厂类 的角色向框架提供 PAGView 实例。业务可通过实现此适配器创建框架的 PAGView 组件，也可以构建自定义 PAGView，再通过 createController 输出实例。

具体实现代码，请参考源码工程 ohosApp 模块的 AppKRPAGAdapter.ets 类。

```

import { IKRPAGViewAdapter, IKRPAGViewController, KRPAGView, KuiklyRenderBaseViewts
} from '@kuikly-open/render';
import { UIContext } from '@ohos.arkui.UIContext';
import { ComponentContent } from '@kit.ArkUI';

class AppKRPAGViewController implements IKRPAGViewController {
    // 实现 play、stop、setProp、addListener、removeListener 等方法
    // ...
}

export class AppKRPAGViewAdapter implements IKRPAGViewAdapter {
    createController(): IKRPAGViewController {
        return new AppKRPAGViewController();
    }

    createPAGView(ctx: UIContext, view: KuiklyRenderBaseView):
    ComponentContent<KuiklyRenderBaseView> {
        return new ComponentContent<KuiklyRenderBaseView>(ctx,
wrapBuilder<[KuiklyRenderBaseView]>(createMyPAGView), view);
    }
}

```